

Question 1

Responsive Web Design (RWD)

Responsive Web Design (RWD) is a web development approach that enables a website to automatically adjust its layout and content according to different screen sizes such as mobile phones, tablets, and desktops. The main goal is to provide an optimal viewing experience without creating separate websites for different devices.

Importance

- Improves user experience on all devices.
- Reduces maintenance by using one website for all screen sizes.
- Increases accessibility and readability.
- Improves SEO because search engines prefer mobile-friendly websites.

CSS Techniques for Responsiveness

1. **Fluid Layouts** – Use percentage (%) instead of fixed pixel widths.
2. **Flexible Images** – Set images with max-width: 100%.
3. **Media Queries** – Apply different CSS rules based on screen size.
4. **Viewport Meta Tag** – Makes the webpage adapt to the device width.

Using Flexbox/Grid

CSS Flexbox and CSS Grid help create adaptive layouts that automatically arrange elements based on available space.

Example (Flexbox):

```
<div class="container">
```

```
<div>Item 1</div>
```

```
<div>Item 2</div>
```

```
<div>Item 3</div>
```

```
</div>
```

```
.container{
```

```
  display:flex;
```

```
    flex-wrap:wrap;
}

.container div{
    flex:1 1 200px;
    margin:10px;
    background:lightblue;
    padding:20px;
}
```

The layout automatically adjusts the number of items per row depending on the screen width.

Improvements

- Use responsive images.
- Increase button size for touch devices.
- Optimize fonts for readability.
- Test the website on different devices and browsers.

Question 2

Client-side Validation

Client-side validation checks user input in the browser before the form is submitted to the server. It provides immediate feedback and reduces unnecessary server requests.

Purpose

- Prevent invalid data submission.
- Improve user experience.
- Reduce server load.
- Save user time.

JavaScript Event Handling

JavaScript events such as submit, input, and blur trigger validation functions.

Role of Regular Expressions (Regex)

Regular expressions are patterns used to verify whether input follows a required format.

Example:

```
<form onsubmit="return validateForm()">
<input type="email" id="email" placeholder="Enter Email">
<input type="submit">
</form>

function validateForm(){
    let email=document.getElementById("email").value;
    let pattern=/^[^\s@]+@[^\s@]+\.[^\s@]+$/;

    if(!pattern.test(email)){
        alert("Invalid Email");
        return false;
    }
    return true;
}
```

A phone number can also be validated using a regex such as:

```
/^[0-9]{10}$/
```

Improvements

- Display error messages beside the input field.
- Use HTML5 validation attributes like required and pattern.
- Always perform server-side validation for security.
- Sanitize user inputs to prevent attacks.

Question 3

CSS Box Model

The CSS Box Model describes how elements occupy space on a webpage.

Components

1. **Content** – The actual text or image.
2. **Padding** – Space around the content.
3. **Border** – Surrounds the padding.
4. **Margin** – Space outside the border.

CSS Positioning Techniques

- **Static** – Default positioning.
- **Relative** – Positioned relative to its normal position.
- **Absolute** – Positioned relative to the nearest positioned ancestor.
- **Fixed** – Stays fixed during scrolling.
- **Sticky** – Switches between relative and fixed.

Causes of Layout Breaking

- Fixed-width elements.
- Large images without scaling.
- Incorrect positioning.
- Overflowing content.
- Missing media queries.

Using Media Queries

```
@media(max-width:768px){  
.container{  
    flex-direction:column;  
}  
}
```

This changes the layout from horizontal to vertical on smaller screens.

Best Practices

- Use Flexbox or Grid.
- Avoid fixed widths.
- Use box-sizing: border-box.
- Optimize images.
- Test layouts on multiple screen sizes.

Question 4

Document Object Model (DOM)

The Document Object Model (DOM) is a tree-like representation of an HTML document. It allows JavaScript to access and modify webpage elements dynamically.

Role in Web Development

The DOM enables developers to:

- Change text and images.
- Modify CSS styles.
- Add or remove elements.
- Respond to user actions.

DOM Manipulation Example

```
<p id="demo">Welcome</p>
```

```
<button onclick="changeText()">Click</button>
```

```
function changeText(){  
    document.getElementById("demo").innerHTML="Welcome to JavaScript!";  
}
```

When the button is clicked, the paragraph text changes without reloading the webpage.

Improvements

- Use event listeners instead of inline events.
- Update only required elements to improve performance.
- Minimize unnecessary DOM manipulation.
- Use asynchronous requests (AJAX/Fetch API) for dynamic data loading.
- Provide loading indicators for better user experience.